MODULE

02

Your first repository

The everyday loop: edit, add, commit. Read history and keep junk out.

YOU WILL LEARN

The three areas	01_three_areas.md
diff & log	02_diff_and_log.md
.gitignore	03_gitignore.md
Exercise: clean commits	ex02_clean_commits/
Interview Q&A	INTERVIEW.md

WHY THE STAGING AREA

Why add before commit?

THE PROBLEM

You fixed a bug, added debug prints and started a new feature, all in the same files. One big commit called **”stuff”** makes the history useless.

THE IDEA

The staging area lets you pick exactly what goes into each commit: one commit per idea.

PICTURE IT THE THREE AREAS

Desk, box, shelf

📦 Your **desk** is the working tree where you edit. **git add** puts things in an open **box** (the staging area). **git commit** seals the box with a label and puts it on the **shelf** (the repository).

working tree
files you edit

→

staging area (index)
the next commit, being packed

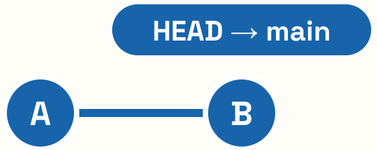
→

repository (.git)
sealed commits, forever

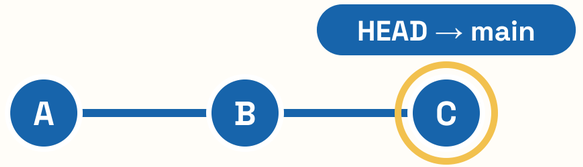
PICTURE IT COMMITS

Every commit adds one snapshot

Before




```
git commit -m "Add cart"
```



The new commit **C** points to its parent **B**, and the branch label **main** (with HEAD) moves forward to it.

TRY IT THE THREE AREAS

Follow one file through the three areas

```
$ git init
Initialized empty Git repository in ~/project/.git/
$ echo "print('hello')" > app.py
$ git status --short
?? app.py
$ git add app.py
$ git status --short
A  app.py
$ git commit -m "Add hello app"
[main (root-commit) 5798825] Add hello app
 1 file changed, 1 insertion(+)
 create mode 100644 app.py
$ git status --short
```

1. **??** untracked: Git sees it, doesn't save it.
2. **A** = added to the staging area.
3. After the commit, the status is clean.

TRY IT PARTIAL COMMITS

Commit only what belongs together

```
$ echo "print('hello, world')" > app.py
$ echo "debug notes" > notes.txt
$ git status --short
 M app.py
?? notes.txt
$ git add app.py
$ git status --short
M  app.py
?? notes.txt
$ git commit -m "Greet the whole world"
[main e49f271] Greet the whole world
 1 file changed, 1 insertion(+), 1 deletion(-)
```

1. Left column = staged, right column = not staged.
2. **notes.txt** stays out of the commit.
3. Interactive picking inside a file: **git add -p**.

TRY IT DIFF

What exactly changed?

```
$ echo "pears = 5" >> cart.py
$ git diff
diff --git a/cart.py b/cart.py
index c8fcf4f..881c683 100644
--- a/cart.py
+++ b/cart.py
@@ -1 +1,2 @@
 apples = 3
+pears = 5
$ git add cart.py
$ git diff --staged --stat
 cart.py | 1 +
 1 file changed, 1 insertion(+)
```

1. **git diff**: changes NOT yet staged.
2. **git diff --staged**: what the next commit will contain.
3. Read your diff before every commit.

STEP BY STEP HISTORY

Reading the log

COMMAND	SHOWS
`git log --oneline`	one line per commit
`git log --stat`	which files changed in each commit
`git log -p -- cart.py`	every change to one file
`git show HEAD`	the last commit: message + diff
`git log --oneline --graph --all`	branches as a picture (Module 4)

TRY IT .GITIGNORE

Keep junk and secrets out

```
$ git status --short --untracked-files=all
?? .env
?? __pycache__/app.pyc
?? app.py
?? server.log
$ echo ".env" > .gitignore
$ echo "*.log" >> .gitignore
$ echo "__pycache__/" >> .gitignore
$ git status --short --untracked-files=all
?? .gitignore
?? app.py
$ git check-ignore -v server.log
.gitignore:2:*.log      server.log
```

1. Patterns: names, ***.log**, folders with **/**.
2. **check-ignore -v** tells you which rule matched.
3. Templates: github.com/github/gitignore.

COMMON MISTAKE .GITIGNORE

Ignoring a file that is already tracked

✗ WRONG

```
git add settings.ini
git commit -m "Add app"
echo "settings.ini" >> .gitignore
# settings.ini is STILL tracked
```

.gitignore only affects **untracked** files. Once committed, a file stays tracked.

✓ RIGHT

```
git rm --cached settings.ini
git commit -m "Stop tracking settings.ini"
# the file stays on disk, Git forgets it
```

git rm --cached removes it from the index only. A leaked secret is still in history: rotate it.

IN REAL JOBS EVERYDAY GIT

The loop you'll run 20 times a day

See where you are

```
git status -sb
```

Review before committing

```
git diff
git add -p
```

Commit one idea

```
git commit -m "Fix rounding in coupon total"
```

Check the result

```
git log --oneline -5
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What is the staging area, and why does it exist?”

- The staging area (index) is the **snapshot being prepared** for the next commit.
- **git add** copies changes into it; **git commit** saves it as a commit.
- It lets you commit **part** of your work: focused commits that are easy to review and revert.
- **git diff** shows unstaged changes; **git diff --staged** shows staged ones.

> **Follow-up they ask next:** ”A file is in .gitignore but still tracked: why?” (it was committed before; use git rm --cached).

RECAP REMEMBER THESE

Module 2 in 5 lines

1. Working tree → **git add** → staging area → **git commit** → repository.
2. **git status -s**: ?? untracked, A added, M modified.
3. **git diff** vs **git diff --staged**.
4. **git log --oneline**, **--stat**, **-p**, **git show**.
5. **.gitignore** for junk and secrets; **git rm --cached** for tracked files.

PRACTICE NOW

Do exercises/ex02_clean_commits/: three focused commits and a .gitignore.